



Temporal metadata analysis: A learning classifier system approach

Michael C. Todd*, Gilbert L. Peterson

Department of Electrical and Computer Engineering, Air Force Institute of Technology, USA

ARTICLE INFO

Dataset link: https://github.com/TheCodeheadMT/dti_prototype

Keywords:

Metadata analysis
Decision support
Learning classifier systems
Automation
Digital forensics

ABSTRACT

Digital forensics is a complex field that requires expert knowledge (EK) and specialized tools to collect, analyze, and report on digital evidence. Temporal metadata analysis is particularly challenging, requiring expert knowledge to understand and interpret underlying traces and associate them with their source. This paper introduces Digital Trace Inspector (DTI), a Learning Classifier System (LCS)-based decision support tool for temporal metadata analysis. DTI leverages a binary Michigan-style LCS to locate and group corroborating temporal digital traces of targeted user activity. Rules are built from expert-created atomics encoded as feature vectors using patterns defined in a structured EK rule framework. The system is evaluated on 10 scenarios of typical user behavior on a Windows 10 workstation. Results show that all models achieved perfect recall, had an average F1 score of 0.98, and required little training data.

1. Introduction

Metadata analysis is a time-consuming and complex process that relies heavily on human experts to apply platform-specific knowledge to identify relevant digital traces. Examiners who study a case's facts form mental models that guide their analysis. Forming these models incurs costs of human time and attention, both limited and in high demand. As backlogs increase, analysis requiring human exploration becomes a significant bottleneck.

In domains characterized by high variability, like computer security, systems based on rule-based signatures can identify patterns using static features within dynamic datasets (Alvarez, 2013; Roesch, 1999; Paxson, 1999). Combining Artificial Intelligence (AI) with rules based on expert knowledge (EK) offers a means to model variability and provide decision support to enhance human decision-making and predictive capabilities for pattern recognition that informs decisions (Nisioti et al., 2021). However, using opaque models that are challenging to explain poses a significant obstacle to their admissibility in legal proceedings (Grimm et al., 2021).

Learning Classifier Systems (LCS) fuse reinforcement learning (RL) with genetic algorithms (GA) and were developed to evolve and apply rule-based systems for problem-solving and decision support (Holland et al., 2000). Michigan-style LCSs (MLCS) are a subset of rule-based ML algorithms that perform well on high-diversity problems and are highly explainable (Urbanowicz and Moore, 2009). MLCs are also effective

with imbalanced datasets and can produce accurate models with little training data.

This paper introduces two novel contributions to temporal metadata analysis of digital forensic data. First, it presents a new method for extracting and encoding machine learning features from semi-structured forensic data using a pattern-matching framework. The YARA-inspired rule structure enables practitioners to create customized features using EK and provides a foundation for training explainable models. Second, it explores how rule-level LCSs can be applied to achieve a generalized approach for temporal metadata analysis decision support and aid examiners in identifying a preponderance of evidence. These contributions are achieved by incorporating work from rule-based learning (Urbanowicz and Browne, 2015) and pattern matching (Alvarez, 2013) into a decision support tool to enhance intelligent automation workflows and identify targeted activity within the metadata.

The remainder of this paper presents an overview of metadata analysis and a methodology for combining tools and techniques to create Digital Trace Inspector (DTI), an automation and decision support tool, and answer the following research questions (RQ):

- RQ1: Can accurate and explainable rule-based models be produced using simulated activities on a Windows 10 Workstation?
- RQ2: How do these models compare to manual Autopsy analysis concerning the required steps and time?

* Corresponding author.

E-mail addresses: michael.todd.11@us.af.mil (M.C. Todd), gilbert.peterson@afit.edu (G.L. Peterson).

Ten training and test datasets are constructed using metadata traces extracted using Log2Timeline (Metz, 2024). In each scenario, atomics based on patterns found in Windows 10 metadata are used along with labeled datasets to train and test MLCS models. Traces are labeled relevant if they are created by actions that, combined with other traces, offer a preponderance of evidence to help the examiner explain what happened during each scenario. Evaluation includes recall, precision, F1 score, and model metrics and compares models built using DTI to using the open-source digital forensic tool Autopsy in analyzing each scenario based on effort measured using time and number of examiner actions.

2. Background

Imagine a scenario in which a marketing manager actively engages in social media, sharing photos on platforms such as Instagram, Facebook, and LinkedIn. They maintain public-facing profiles due to the personal and professional benefits of networking. One day, they receive a distressing email with the subject “URGENT: Pictures of you,” claiming possession of explicit photos that could damage their career. The email demands \$5,000 to a Bitcoin address within one week, threatening to send the pictures to their contacts, including their employer if the ransom goes unpaid. Distressed, the manager considers paying but instead contacts the authorities.

In such a scenario, the perpetrator’s device would likely contain temporal metadata traces critical for substantiating allegations of cyber extortion, identity theft, and harassment. The perpetrator’s digital activities—downloading files, interacting with web applications, and modifying images—leave behind a metadata trail. Forensic experts analyze this data to build a timeline of events, supporting or refuting the allegations through temporal metadata analysis.

The next logical step for an examiner is to review all metadata traces to determine if any are relevant to the allegation being tested. This process requires simultaneously identifying which traces are created by each action and determining if they are relevant to the current case. Since determining relevance is highly subjective and may depend on latent factors, the proposed system leverages EK to provide practitioners with a mechanism to automate the discovery and grouping of traces associated with actions.

2.1. Temporal metadata

Metadata is a widely recognized term in digital forensics, and it commonly refers to data that provides descriptive information about other data. Meta (Merriam-Webster, 2024) originates from the Greek prefix meaning “after” or “beyond”, and does not encompass the actual content of the primary data. Metadata is often used to support the provenance of a digital artifact and aids forensic experts in reconstructing events and verifying the authenticity of evidence.

For example, consider a picture discovered on a suspect’s device. Metadata encompasses information included with this file, including file-specific details, timestamps, camera-related data, user comments, digital signatures, and modification history. It also includes operating system information such as system logs, registry data, system configuration, memory data, network activity, user account data, and hardware and application data. When this metadata is extracted and examined, it assumes the role of a digital trace, which investigators carefully scrutinize to glean insights for the investigative process. Some examples of information captured in metadata include:

- Modify, Access, Create, Born (MACB) times of objects
- Upload, download, and delete times of objects
- Transfer times of objects to/from external media
- Application events times (start, stop, connections)
- System and event user times (login/logout)

Operating systems enable pseudo-concurrency through processes, where each process shares CPU time to virtually run simultaneously (Tanenbaum, 2009). Concurrent processes complicate extracted metadata because running programs overlap and produce interleaved traces. Since all operations are not cataloged, processes often modify media without leaving metadata traces. When traces overlap and interleave, they are considered partially ordered and, without context, challenging to group. Many effective methods exist to process and interpret temporal metadata, including data mining, visualization, and modeling.

Sequential pattern mining is identifying patterns that occur in a specific order within a database of sequences where a pattern is considered significant if it appears in at least a user-defined minimum number of sequences. (Agrawal and Srikant, 1995) Previous research on mining patterns from interleaved metadata (Okolica, 2017; Okolica et al., 2020) using temporal sequence mining has demonstrated that patterns can be identified by extracting terms and sequences to create production rules, reducing datasets by 96%. Villafane also studied temporal data that overlaps and showed that some datasets are better understood when viewed using events as intervals (Villafane et al., 2000). They discuss how other events often contain events and how this relationship may provide important context. Though effective, these methods do not provide an explainable framework for practitioners to build custom models for their unique needs.

Traditionally, examiners study temporal metadata using open-source and commercial tools like Autopsy (Carrier, 2013), TimeSketch (Berggren and Peterson, 2023), and EnCase (Guidance-Software, 2023). Customized filtering options exist in these tools but focus primarily on presenting everything possible to the examiner in the most efficient way possible. Unfortunately, with modern computers capable of creating millions of traces a second, manual analysis of every trace is intractable.

One approach to this challenge is to create visualizations of timeline data like Adderley’s TAIMA (Adderley and Peterson, 2020) or those provided in Autopsy and EnCase. However, these abstractions exclude expert knowledge that is useful when searching for specific behaviors or events. While tools like Metz’s Log2Timeline (Metz, 2024) effectively organize and extract traces from multiple platforms, including Windows, Linux, MacOS, and Android, they lack decision support to assist examiners with evolving platforms and growing backlogs. Timeline2GUI (Debinski et al., 2019) is another recent tool that provides a query-based approach to assist users in discerning traces using a color-coded system. This approach provides limited decision support by categorizing results using keywords or boolean conditions, but does not correlate and group traces using EK.

2.2. Decision support

Decision support uses algorithms to enhance human decision-making and provide insights or predictions to aid in pattern recognition, risk assessment, and other applications (Shim et al., 2002). A decision support system (DSS) aims to automate specific decision processes and improve the accuracy and efficiency of human judgments. To our knowledge, the proposed methodology has not been used before to provide decision support to temporal metadata analysis. Existing frameworks for aiding digital forensics can be categorized into reasoning support and efficiency, correlation and attribution, and soundness of forensic evaluation (Nisioti et al., 2021).

Rule-based signature models are often preferred over opaque models like neural networks in legal proceedings due to their interpretability, consistency, and transparency, which are crucial for legal acceptability (Grimm et al., 2021). These systems operate on clear, predefined rules, making their decisions easily understandable and reliable. They are often less complex and more accessible to validate, avoiding the computational intensity and “black box” nature of deep learning.

Turnbull (Turnbull and Randhawa, 2015) proposed a reasoning support and efficiency approach *ParFor* with a forward-chain rule-based reasoning system to abstract low-level data into information that is

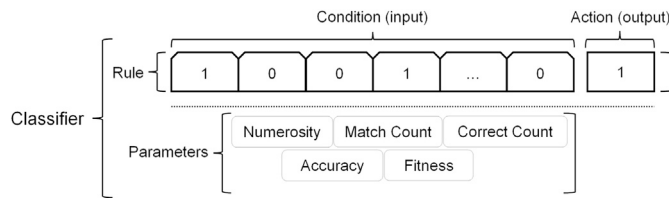


Fig. 1. Classifiers are the basic learning unit of an MLCS, used to form a learning population P and as parents to produce offspring, evolving the rule set each iteration using GA operations.

more useful for analysts to consider. They applied standardized knowledge representation techniques and automated, rule-based systems to encapsulate EK for analysis. The primary focus was to automatically derive ‘events’ from base forensic artifacts, such as system events (logins, startups, shutdowns) and user events (web browsing, sending emails). However, this approach relies on ontological mappings and does not allow a user to customize a model for later use.

Another reasoning support approach is to use Answer Set Programming (ASP) proposed by Costantini (Costantini et al., 2019). This approach posits complex investigations as optimization problems within the ASP framework. They show that realistic investigative scenarios can be modeled as an ASP program that can produce plausible hypotheses. However, real-world applications are provisional, with only a rudimentary sketch of a decision support system tailored to aid digital forensics investigators.

A data-driven approach that suggests actions to a researcher based on observed attack actions using the MITRE ATT&CK framework was proposed by Nisioti (Nisioti et al., 2021) to leverage community-supported tactics, techniques, and procedures to assist in incremental decision-making. This approach requires a framework like MITRE ATT&CK as a reference to make decisions, and no such framework exists for temporal metadata analysis.

2.3. Learning classifier systems

LCSs can be applied to supervised and unsupervised learning problems across various domains. One example is using an LCS to create an intelligent binary email classifier, e.g., spam and non-spam emails. The four universal components of LCSs are a finite population (set) of classifiers P that represents the knowledge base and parents for future offspring (described in Fig. 1); a *Fitness* component used to determine if a rule is fit enough to be kept in the final population; a reinforcement component to assign credit to well-performing rules; and a discovery component to find new rules that may perform better (Urbanowicz and Browne, 2015). This study applies a hybrid supervised-reinforcement learning approach, starting with labeled rules and then leveraging reinforcement learning to explore the entire problem space.

LCSs use the environment, a learning component, and rule compaction to generate rules to explore a problem space. The environment comprises observations captured in attributes (the *Condition*) represented in a binary vector. After receiving input, incremental learning occurs on a rule-by-rule basis, where a rule is a set of factor *Conditions* (input) associated with an *Action* (output or class). When a rule is instantiated, it becomes a classifier with specific *Parameters* and acts as the basic unit for learning within an LCS as shown below in Fig. 1. Once a user-defined number of iterations are completed, a post-processing step called rule compaction is performed to eliminate poor-performing or redundant rules. (Urbanowicz and Browne, 2015).

Michigan-Style Learning Classifier Systems (MLCS) are a subset of rule-based ML algorithms that evolve new classifiers (rules) using a GA through the combination and *Fitness* of existing rules (Urbanowicz and Moore, 2015). In this study, the Extended Supervised Tracking and Classifying System (ExSTraCS) MLCSs is applied because it performs well on problems involving high diversity (heterogeneity) and epistasis (Urbanowicz et al., 2012), both common in temporal metadata analysis. In

this context, diversity arises from the complex states created by pseudo-concurrency in processes, such as overlapping and interleaved traces. Additionally, epistasis—where one feature may suppress or mask the effects of another—plays a critical role in decision-making during the analysis. Starting with a set of rules known as population classifiers, the goal is to evaluate observations from a domain under inspection and learn new rules by applying a GA. The desired end state is a mechanism to accurately make decisions (classify) members in the form of rules from the evolved population.

Parameters used to determine the *Fitness* of a given classifier are (Urbanowicz and Browne, 2015):

- *Numerosity*: a scalar representing the number of copies of a rule (classifier) in the population.
- *Match Count*: a scalar representing the total number of rules in the set that match the current training observation, divided into correct and incorrect sets.
- *Correct Count*: a scalar representing the number of rules in the correct set.
- *Accuracy*: a quotient of correct count and match count calculated as $Correct\ Count / Match\ Count$.
- *Fitness*: a function of accuracy to represent the utility of a classifier using power parameter v to determine the importance of high accuracy when calculating *Fitness* as $Accuracy^v$.

$$Fitness = Accuracy^v = \left(\frac{Correct\ Count}{Match\ Count} \right)^v$$

3. Methodology

The application of MLCSs for labeling temporal metadata traces using Digital Trace Inspector (DTI) encompasses the five steps shown in Fig. 2. First, digital media is acquired, and events are extracted into a collection of semi-structured (CSV) traces using the Plaso Log2Timeline (Metz, 2024). Next, expert knowledge rules are created to abstract intervals and characteristics using a YARA-inspired format (Alvarez, 2013) to match patterns found in the metadata traces. The third step requires a mechanism to evaluate conditions in EK rules against the metadata to form binary encoded feature vectors for each trace. Next, an MLCS trained using labeled feature vectors to generate a set of solution rules. Finally, trained models predict the class of unseen traces encoded using the same EK rules.

The generalized approach to applying the DTI system includes the following steps and overlaps with steps one through five in Fig. 2:

1. Select and aggregate fields from desired sources
2. Leverage and/or create platform-specific EK rules
3. Create targeted activity EK rules and label traces where targeted activity is class ‘1’ and all others class ‘0’
4. Train a model using EK rule encoded training data
5. Predict labels of unseen datasets encoded using the same EK rules

To demonstrate the implementation and effectiveness of the DTI system on a specific platform, datasets are compiled to simulate activities resembling those in a cyber extortion scenario. The task involves learning labels for traces left by specific user interactions, focusing on the simulated actions within each dataset. In these scenarios, user activities include downloading, modifying, moving, deleting, and transferring files to various locations using web applications and a USB device.

Formal definition: DTI applies a binary classifier MLCS to evolve a set of encoded digital traces as relevant or irrelevant based on EK rules provided by an expert. Let T be a set of labeled traces $t \in T$, such that $t = (datetime, field_1, field_2, \dots, field_n, class_label)$.

Each atomic feature represents *interval* and *characteristic* abstractions for a specific platform and forms the set of all EK rules $e \in E$ such that $i = |E|$, and $e = (feature_name, conditions)$. Using T as input, features are

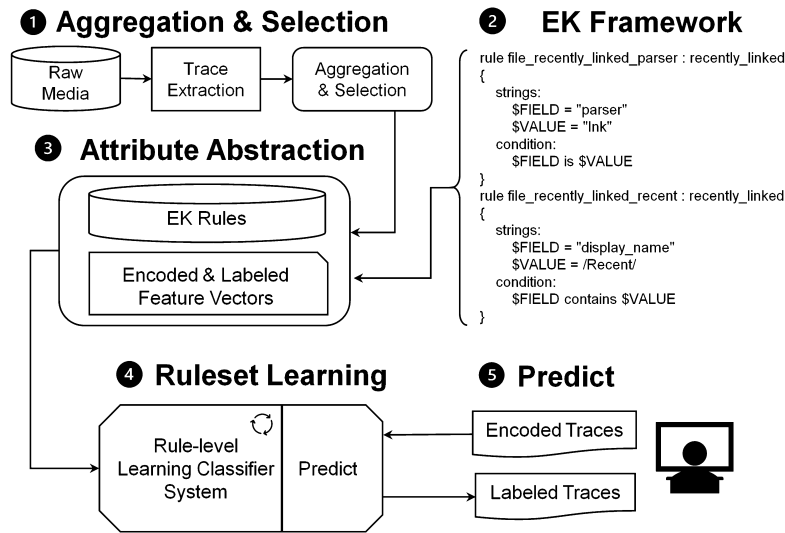


Fig. 2. Digital Trace Inspector system flow: (1) Aggregate and select fields from labeled raw data to form T ; (2) Create EK rules focused on targeted activity to form E ; (3) Encode all traces using EK rules to form F ; (4) Train a rule-level binary classifier with encoded feature vectors to produce $\mathcal{L}$; (5) Predict relevance on unseen feature vectors using $\mathcal{L}(F_{new})$.

extracted to form F , a list of binary encoded labeled feature vectors $f \in F$, such that $f = (f_1, f_2, \dots, f_{i+1})$, $|f| = i + 1$, and $f_i^{(m)}$ is the i^{th} value of the m^{th} feature vector, where the first i elements represent the condition EK rule and the final element is the class. For each feature vector $f^{(m)}$ and each $e \in E$, if conditions in e_i are met, f_i is 1, and otherwise 0. Finally, the binary classifier $\mathcal{L}$ uses F as the starting population to learn the label of the entire 2^i problem space, i.e., relevant or not.

Actions & Events: When reasoning about metadata, the concepts of actions and events help examiners understand the information captured. This study considers an action as the base-level abstraction representing a collection of processes producing temporally aligned traces. An action is a step a user takes that leaves behind data or “traces”. Traces are the byproduct of the operations carried out during actions. For example, when a user opens an image file in photo editing software, the MACB, registry, logs, and temporary files may update and produce traces.

As a higher-level abstraction, events are composed of one or more actions that represent human-understandable concepts. An event is a meaningful grouping of actions that together are significant. For example, in the case of cyber extortion, when the perpetrator interacts with web applications like Facebook, the session can be considered an event comprising various actions like searching, adding, downloading files, and clicking links. Each of these actions contributes traces that, when combined, form a comprehensive picture of the event.

To abstract meaning from underlying metadata and identify events, DTI employs EK rules designed using the following principles:

- *Characteristic* abstractions can be formed using patterns observed in traces created by actions.
- *Interval* abstractions can be formed by identifying patterns within traces created by actions to signify the start and stop of an interval.
- Events can be identified using *interval* and *characteristic* abstractions.

3.1. Aggregation and selection

Managing semi-structured datasets requires slicing (selecting subsets of rows or columns), dropping, renaming, and concatenation operations. Fig. 3 illustrates how the DTI system facilitates the selection and aggregation process using a list of raw source files R , a user-defined field definition $fields$, and a configuration field mapping M . Each mapping configuration consists of a *key:value* pair where the *key* matches a field in $fields$, and *value* matches a field in one or more of the sources $r \in R$.

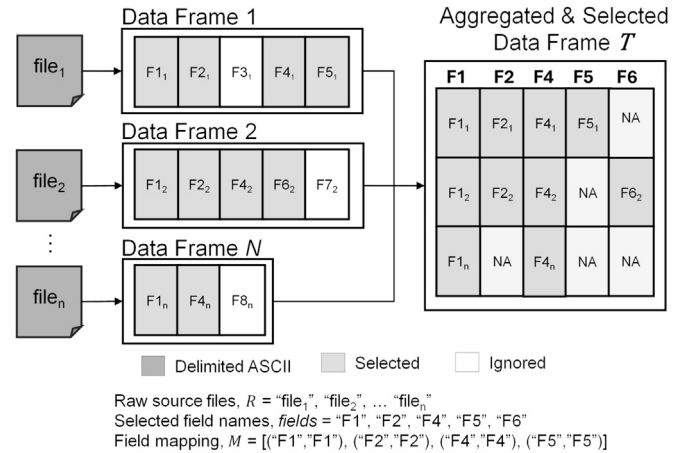


Fig. 3. Aggregation and selection of fields from ASCII delimited raw sources.

Each source file is parsed, with columns matching selected headers kept and all others dropped. ‘NA’ is inserted if data sources do not contain all of the fields specified in $fields$. Each r is concatenated and sorted by *datetime* and produces T , an $m \times n$ data structure of m traces with n fields.

For instance, suppose a laptop computer and phone are seized during a search warrant. After processing with data collection and extraction tools, assume each device produced a semi-structured ASCII delimited set of temporal metadata: r_1 (laptop) with fields *datetime*, *message*, and *source* and r_2 (phone) with fields *datetime*, *text_msg*, and *app*, where $|r_1| = p$ and $|r_2| = q$. To combine and temporally align events from the laptop with phone messages, let field definition $fields = [datetime, laptop_msg, phone_msg]$, and mappings $M = [(datetime:datetime), (laptop_msg:message), (phone_msg:text_msg)]$ to create a single dataset with three columns and $p + q$ rows. Note how fields *datetime*, *laptop_msg*, and *phone_msg* are selected and aggregated from r_1 and r_2 but fields *source* and *app* are dropped.

3.2. EK framework

Existing rule-based systems like YARA (Alvarez, 2013), Snort (Roesch, 1999), and ZEEK (Paxson, 1999) are widely adopted and successful at pattern-matching tasks because they are flexible and under-

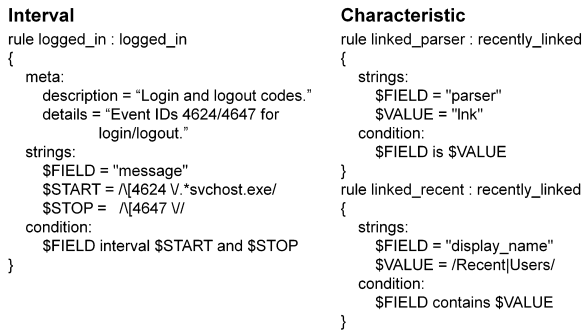


Fig. 4. Interval and characteristic EK rule structure.

standable. Additionally, these rules can be quickly modified and shared with a team or community. Due to these properties and previous success in application, this study adopted a rule-based framework inspired by YARA rules.

Fig. 4 shows the structure for EK rules. All rules begin with “rule” followed by a rule name and feature name separated by a colon. The section *meta* is an example of an optional section in the rule structure. Rules require two sections: *strings* and *condition*. The *strings* section contains variables \$FIELD, and patterns are defined under \$START and \$STOP for *interval* abstractions, or \$START, \$STOP, and \$VALUE for *characteristic* abstractions. *Condition* contains a single expression composed of terms defined in *strings* and a supported operator.

Fig. 4 shows *interval* rule *logged_in* that defines patterns for when a user logs in (\$START) and logs out (\$STOP). *Intervals* are limited to only a single rule but can capture multiple start and stop patterns using regular expressions. In contrast, *characteristic* rules can contain more than one rule to create compound conditions. Compound conditions are evaluated using a logical AND operation. For instance, *characteristic* abstraction *recently_linked* is formed using two rules—*linked_parser* and *linked_recent*, where both must be *True* for the feature to evaluate as *true*. In this example, the \$VALUE of the second rule contains regex “/Recent|Users/”, and is parsed and evaluated as the following expression:

$$(\$FIELD \text{ is 'display_name'}) \wedge (\$FIELD \text{ contains 'Recent'} \vee \$FIELD \text{ contains 'Users'}).$$

Due to the nature of metadata, analysts often face the paradox of having too much data but insufficient information. Adding value to data to create meaningful information is complicated by the temporal interleaving of processes, where different events occur concurrently. This vast and complex intertwining of data can render an unfiltered or unlabeled timeline overwhelming, even for skilled examiners. Two factors contributing to this complexity are *overlapping states* with similar traits and *unknown states*.

Complexity occurs because of *overlapping states* when data is organized chronologically and more than one event is executed concurrently. For example, application A and B are executed one after the other and continue running for an unknown duration. Concurrent processes create partially ordered and *interleaved* traces that are intermingled. Extending the example above, applications A and B create temporally interleaved traces that cannot be linked to either program without additional information, expert knowledge, or both. Additionally, *unknown state* occurs when the system state is underreported in metadata. Underreporting occurs for many reasons, including obfuscation, design decisions, efficiency, coding errors, collection tool limitations, and storage and processing constraints. Often, these decisions are made to limit I/O delays and avoid overloading storage media with unnecessary information. Ultimately, many existing tools depend on the examiner to disentangle traces and identify patterns within metadata, particularly when reasoning about overlapping events, interleaved traces, and ambiguous states.

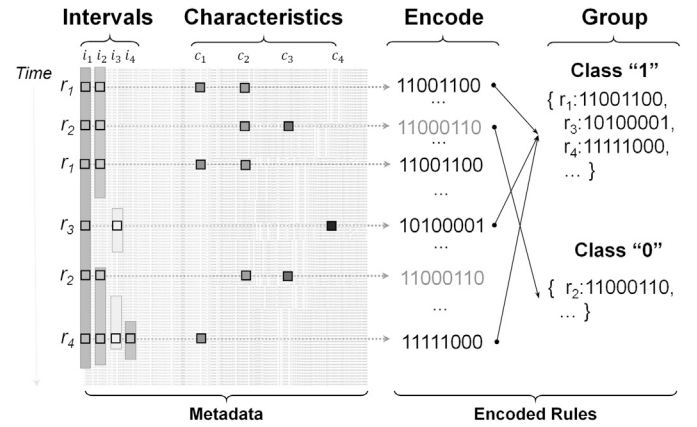


Fig. 5. Overlapping state discerned using *interval* (vertical bars) and *characteristic* (squares) features to represent states with and without a duration. A black outline indicates pattern matching conditions for the associated EK rule are *True*.

Nevertheless, examiners routinely identify patterns in metadata that may uncover important information. The proposed EK rule framework uses these patterns as ASCII strings or regular expressions to identify specific features to create two types of abstraction, *intervals* and *characteristics*. These abstractions represent an association between EK rules and digital traces using operators that support exact matching (*is*), partial matching (*contains*), and comparison (between, greater than (*gt*), less than (*lt*), greater than or equal (*ge*), less than or equal (*le*), equal (*eq*), and not equal (*ne*). All rules employ the string \$FIELD to identify the field within each trace that is being tested. If the association defined in an EK rule is *True*, trace *t* is said to be *associated* with EK rule *e*. An *association* is represented using the commutative * operator, e.g., A associated with B is $A * B$.

Interval abstractions represent an association with a duration, defined by start and stop patterns, and include one operator, *interval*. These rules represent a change in the system state, e.g., an application with a duration started and stopped or a connection begins and then ends. *Interval* abstractions are formed using \$START and \$STOP patterns where the period begins and ends, respectively. Patterns are tested against *t* to calculate *f*, where $f_i^{(m)} = 1$ if $\text{datetime}(\$START * \$FIELD) \leq \text{datetime}(t) \leq \text{datetime}(\$STOP * \$FIELD)$, and otherwise 0.

Characteristic abstractions represent an association between EK rules and traces without an implied duration. They are used in concert with *interval* rules or, when used alone, are functionally equivalent to a string search. *Characteristic* abstractions are formed using a \$VALUE pattern and *t* to calculate *f*, where $f_i^{(m)} = 1$ if $\$VALUE * \$FIELD$, and otherwise 0.

These abstractions enable examiners to combine their expertise with the EK framework to create atomics for a specific platform to train a learning machine. The purpose of these rules is to provide an explainable and intuitive method for identifying and labeling complex states observed in metadata. For example, Fig. 5 illustrates the application of four *intervals* and *characteristic* features to differentiate between events with overlapping states and interleaved observations. In this illustration, *intervals* are vertical bars extending over time, covering one or more traces. Note how i_1 extends nearly the entire set of metadata, whereas i_4 only occurs once later in the dataset. In contrast, *characteristics* are confined to the specific instance where the associated pattern is matched and do not extend beyond that point. By combining *intervals* and *characteristics*, learned rules can discern complex patterns in metadata, such as events that overlap (occur simultaneously and are unrelated) or interleaved (occur in a sequence but are unrelated). This approach facilitates a nuanced and flexible method for finding meaningful patterns in metadata. Table 1 shows an example of rules learned by a classifier where encoding for r_1 and r_2 are interleaved but are in different classes. One key feature of MCSLs is a *subsumption* mechanism that reduces the over-

Table 1
Encoded feature vectors and learned rules for Fig. 5.

Rule	Feature Vector	Learned										Class
r_1	11001100 1	1	1	0	0	1	1	0	0	1		
r_2	11000110 0	1	1	0	0	0	1	1	0	0		
r_3	10100001 1	1	0	1	0	0	0	0	1	1		
r_4	11111000 1	1	1	1	1	1	0	0	0	1		
$r_{(34)}$	Using r_3, r_4	1	#	1	#	#	0	0	#	1		

all solution (model) size by replacing rules with the most generalized version possible. In Table 1, we see an example of *subsumption* where rules r_3 and r_4 are replaced by a single generalized rule $r_{(34)}$ where the # represents the feature being unspecified (either 1 or 0).

Three factors limit abstraction using this approach. First, identifying patterns in metadata traces is necessary, as feature abstraction is impossible without expert knowledge. Further, pattern matching using a rule-based framework is subject to human error, e.g., typos, mistakes, or incorrect patterns. Lastly, ambiguous traces without unique *characteristic* patterns are difficult to distinguish. For instance, NTFS update sequence number (USN) journal entries in the studied datasets provide a file handle, reference, parent file, update source, and update reason but no further context. These traces are largely indiscernible even when combined with *interval* rules.

3.2.1. Rule-building example

To illustrate how to apply the EK framework in practice, we create a simplified rule set for Windows 10 to correlate the metadata of a user uploading a .jpg to a Google Drive. This example applies steps two and three of the generalized process to *leverage and/or create* platform-specific rules focused on targeted activity. The number of features in this example is reduced to demonstrate the rule-building process succinctly. In the following scenario, a user logs in, opens a browser, and uploads a file to Google Docs. After studying all traces and labeling the targeted activity, eight patterns are identified, two *intervals* and six *characteristic*.

First, two *interval* rules are listed below based on \$START and \$STOP patterns identified when a user logs in and out and when Chrome is opened and closed. In the former, event IDs 4624 and 4647 are distinct start and stop patterns, and in the latter, strings '[CHROME.EXE] was executed' and '[59 Chrome Component Update]' appear to represent the start and stop of an *interval*. These patterns are used to develop the following EK rules:

```
rule logged_in : logged_in
{
  strings:
    $FIELD = "message"
    $START = /\[4624 \\.svchost.exe/
    $STOP = /\[4647 \//
  condition:
    $FIELD interval $START and $STOP
}

rule chrome_running : chrome_running
{
  strings:
    $FIELD = "message"
    $START = /\[CHROME.EXE\] was executed/
    $STOP = /\[59 \\.Chrome Comp.*Update/
  condition:
    $FIELD interval $START and $STOP
}.
```

Next, we focus on what the user typed and the web application Google Docs. This activity implies that a web browser is running ('CHROME.EXE'), the user interacts with the browser ('User typed' or

'User clicked'), and the web address for Google Docs ('drive.google.com') is visited. These features are captured with the following EK rules:

```
rule browser_app : browser_app
{
  strings:
    $FIELD = "message"
    $VALUE = /\[.*CHROME.EXE.*\].*executed/
  condition:
    $FIELD contains $VALUE
}

rule usr_typed_clicked : usr_typed_clicked
{
  strings:
    $FIELD = "message"
    $VALUE = /User typed.*\|
      User clicked.*\|/
  condition:
    $FIELD contains $VALUE
}

rule webapp_google_drive : webapp_google_drive
{
  strings:
    $FIELD = "message"
    $VALUE = /drive.google.com/
  condition:
    $FIELD contains $VALUE
}.
```

The last three rules employ patterns concerning link files and registry keys created during the targeted activity. These rules include referencing the most recently used files (MRUList), .lnk files created under the *Users* directory, and traces containing .lnk file references. These features are extracted using the following *characteristic* abstractions:

```
rule mru_list_regkey : mru_list_regkey
{
  strings:
    $FIELD = "source_long"
    $VALUE = "MRUListEx Registry Key"
  condition:
    $FIELD is $VALUE
}

rule usr_lnk_ref : usr_lnk_ref
{
  strings:
    $FIELD = "message"
    $VALUE = /\Users.*.lnk/
  condition:
    $FIELD contains $VALUE
}

rule ntfs_lnk_ref : ntfs_lnk_ref
{
  strings:
    $FIELD = "message"
    $VALUE = /.lnk File reference:/
  condition:
    $FIELD contains $VALUE
}.
```

Using the DTI prototype, these eight EK rules can be applied to encode metadata traces and train a model with eight features (attributes)

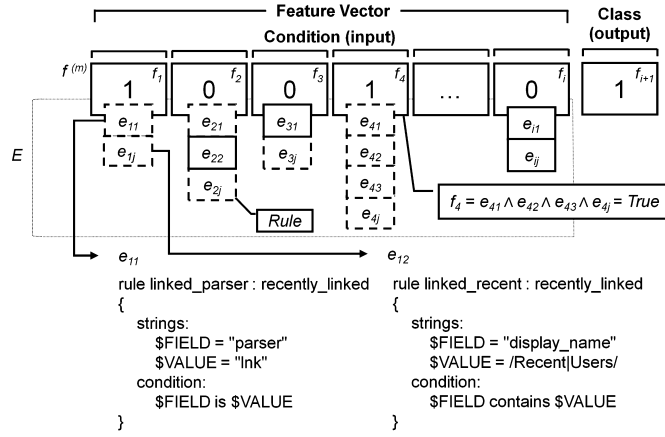


Fig. 6. Feature vector attribute encoding process for each trace using EK rules.

with a problem space of 2^8 possible feature vectors. Additionally, the eight EK rules described above were created during the study of dataset S5 and represent a ‘reduced’ rule set, a subset of the 55 Windows 10 EK rules used to train and evaluate models later in this study.

3.3. Attribute abstraction

Since each rule may contain multiple patterns to check for each condition, generating feature vectors can be computationally expensive. As a result, all calculations outside of flow control are vector operations. Binary encoded feature vectors for each trace are calculated using *pandas* (McKinney, 2010) and *numpy* (Oliphant, 2006) vector operations to evaluate boolean expressions across large datasets efficiently.

Fig. 6 illustrates feature vector $f^{(m)}$, where trace $t \in T$ is encoded using EK rules in E . Conditions for each feature are listed in e_{ij} , where $i = |E|$ and j varies with the number of conditions. The state for each $f_i \in f^{(m)}$ is calculated by applying a logical AND operation across all j conditions.

3.3.1. Intervals

Interval abstractions imply a duration and are calculated using Algorithm 1 and further illustrated in Fig. 7, where trace indexes are matched using starts and stops to create index-bounded intervals. This process iterates through \$STOP indices by comparing indexes to determine if *current_stop* follows *start*. If the *current_stop* occurs before *start*, the next stop is checked until *start* occurs before *current_stop*. Then *start* and *current_stop* are added as paired starts/stops. This approach was selected to ensure multiple instances of an event can be terminated using a single-stop pattern. For instance, note how stop index 80 is not included as a stop. This mechanism introduces the potential for terminating an *inter-*

val before the last \$STOP pattern occurs, so \$STOP patterns should only indicate the absolute termination of an interval.

Algorithm 1 Pair start and stop patterns.

Require: *starts* list of indexes with start pattern found

Require: *stops* list of indexes with stop pattern found

Ensure: *paired_starts* ← new list

Ensure: *paired_stops* ← new list

```

1: stop_iter ← iter(stops)
2: current_stop ← next(stop_iter)
3: if current_stop is None then
    return (paired_starts, paired_stops)
4: end if
5: for start ∈ starts do
6:   while current_stop < start do
7:     current_stop ← next(stop_iter)
8:   end while
9:   paired_starts.append(start)
10:  paired_stops.append(current_stop)
11: end for
    return (paired_starts, paired_stops)

```

3.3.2. Characteristics

Characteristic abstractions are calculated using vector operations by applying \$FIELD, \$VALUE, and one of the operators listed in Section 3.2. No further processing is required since a *characteristic* feature has no implied duration. Fig. 6 shows two EK rules e_{11} and e_{12} are used to create a feature that tests if both conditions are *True*.

3.4. Rule set learning

As a supervised binary classifier, DTI produces models that locate and group corroborating traces that provide context to an analyst for decision-making tasks. DTI reads a labeled set of traces and uses EK rules to encode labeled traces to create a population of classifiers and explore the problem space, retaining only those with the highest *Fitness*. After a set number of iterations, DTI forms a solution rule set (model) to predict a label for any possible feature vector (encoded trace) using the same EK rules. When training each ExSTraCS MLCS, although several parameters can be adjusted based on the problem domain (Urbanowicz and Moore, 2015), only the parameters listed in Table 2 are exposed in the DTI prototype.

The ExSTraCS learning process integrates RL and GA operations, making the number of iterations a critical hyperparameter for achieving a comprehensive solution. Without a sufficient number of iterations, the system may fail to learn labels for some feature combinations. This study set the number of iterations to 100,000 for two key reasons. First, prior research using ExSTraCS to solve the 70-multiplexer and 137-multiplexer problems recommended using the hyperparameter settings

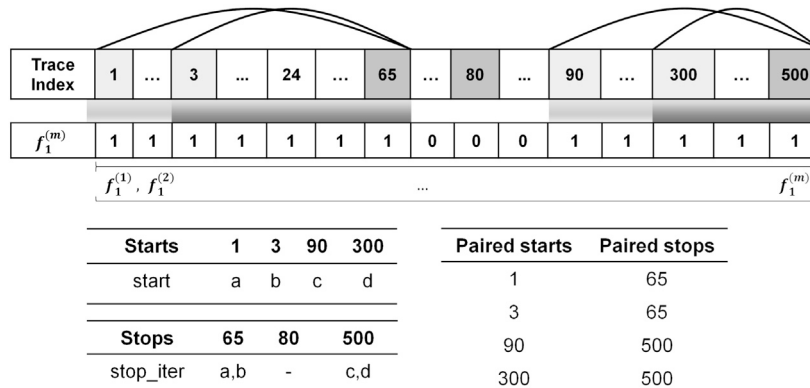


Fig. 7. Calculating an interval using Algorithm 1 where a single stop pattern terminates multiple start patterns.

Table 2
DTI API exposed ExSTraCS MLCs hyperparameters.

Parameter	Value	Description
iterations	100,000	The number of training cycles (iterations)
N	6,000	Maximum micro classifier population size (sum of classifier numerosities)
ν (ν)	5	Power parameter used to determine the importance of high accuracy when calculating <i>Fitness</i> as (<i>accuracy</i> ^{ν})

Descriptions from (Urbanowicz and Moore, 2015).

outlined in Table 2 as a starting point (Urbanowicz and Moore, 2015). Second, even if accuracy stabilizes before reaching 100,000 iterations, a more general model is preferred, as it yields a smaller and more generalized rule set.

A micro-classifier population size N of 6,000 was chosen to set the total number of rules permitted in the final solution set. The micro-classifier population is the sum of the *Numerosities* of all population members. This parameter ensures that only classifiers with the highest utility remain in the population. The suggested value of 6,000 is set because any training errors found during analysis were unrelated to population size, and adjusting this parameter did not impact accuracy or training time.

The final parameter exposed by DTI is the accuracy power parameter ν . Increasing ν emphasizes accuracy by narrowing the population to include only members with the correct labels. As a result, higher values of ν reduce the number of iterations needed to learn the correct labels for the problem space. This study selected $\nu = 5$ because it provides accurate models across all scenarios within the chosen iteration limit. While increasing ν beyond 5 does yield the same accuracy in fewer iterations, all models were trained for 100,000 iterations to promote generality.

3.4.1. Bootstrapping MLCs

Oversampling with replacement is a statistical bootstrapping technique that uses existing observations from a specific class to create a more accurate estimator (Efron and Tibshirani, 1994). This technique can create a balanced dataset when the minority class is oversampled with replacement. Orriols (Orriols and Bernadó-Mansilla, 2005) studied the class imbalance problem in LCSs and suggested classifier system and sample-level approaches to dealing with imbalanced datasets for UCS, a version of XCS classifier system (Wilson, 1995). For MLCs, oversampling causes the match and correct count of the minority class to increase, increasing the accuracy and *Fitness* of these individual classifiers.

For example, in a population with n features, 2^n binary strings are possible in the problem space. Suppose F is a list of labeled binary feature vectors. As training begins, each $f \in F$ is added as a classifier to population P , and if already present, its *Numerosity* is incremented. Thus, oversampling only impacts the parameters of classifiers resampled in the solution rule set. However, due to the survival of the fittest and pruning mechanism, boosting the *Fitness* of some classifiers may cause others to drop from the population.

Recall that supervised MLCs are trained using labeled feature vectors, in this case, a combination of ones and zeros. During this study, all training datasets are bootstrapped by *oversampling with replacement* of the minority class (class 1) to achieve a balanced dataset. For instance, after balancing, the training dataset for $S1$ has a class distribution ratio of 1:1, with 10,468 in both classes, for a total dataset size of 20,936. Though not implemented in this initial study, strategies to reduce the training dataset to unique feature vectors first and then oversample both classes are also viable options.

4. Evaluation & results

DTI is evaluated using two experiments. The first tests if accurate models can be produced using simulated activity on a Windows 10 work-

Table 3
EK rules added by scenario.

S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
14	6	7	4	5	3	2	5	7	2

station, and the second compares manual analysis using the open-source tool Autopsy to the model produced using DTI. Models are trained and tested using a Windows 11 workstation with processor i9-11,900K @ 3.5Gz and 32G RAM. Analysis with Autopsy 4.19.3 is performed on a Windows 10 workstation virtual machine with 64G RAM and 1TB storage. Training and test collection workstations are virtual machines running Windows 10, 8G RAM, and 20G storage. Comparison experiments are recorded and reviewed to measure time and number of steps.

Models for each scenario are trained and validated using steps 1-4 of the DTI system flow described in Fig. 2. As each set of labeled traces is studied, EK rules (atoms) are defined by identifying patterns in targeted traces, following the rule-building process outlined in Section 3.2.1. These rules are developed sequentially across scenarios from $S1$ to $S10$, with the potential for rules created in earlier scenarios to benefit later ones. For instance, the rule-building example targets Google Drive file upload activity and requires eight rules. Three rules (logged_in, chrome_running, user_typed_clicked) exist from the EK rules created in $S1$. This cumulative approach allows for cross-scenario rule application, increasing the generalizability and utility of the model across different situations. During this process, 55 EK rules are created and shown in Table 3.

$S1$ required the most EK rules because it encompassed several patterns found in other scenarios. Subsequent scenarios required fewer rules as they focused more on the specific patterns tied to the targeted activity. The EK rules developed during this process are crucial for identifying relevant and irrelevant attributes, refining the learned rules, and training an accurate model. Features whose EK rule patterns are not found in any trace within the training data are excluded to facilitate the evaluation of building models with different feature sizes.

After creating all EK rules, models are trained and assessed using 5-fold cross-validation. A model is accepted if the CV score reaches 1.0. If the score falls short, EK rules are modified until the desired score is achieved. Testing each model follows step 5 of the DTI system flow described in Fig. 2. For each scenario, models are evaluated using Precision, Recall, F1 scores, and additional model metrics. These metrics include approximate accuracy (the ratio of correct classifiers to the total number of classifiers), average generality (the ratio of the average proportion of unspecified attributes (#) in rules to the total population), learned rules, macro population size, and micro population size. Generality assesses how well the model maximizes generality while balancing accuracy. Since the problem space doubles for every feature added, generality is important to ensure the final solution (model) is as small as possible.

4.1. Datasets

Two datasets are constructed using Windows 10 workstations and acquired by cloning each virtual machine's virtual machine disk (VMDK). Metadata traces are extracted using plaso Log2Timeline version 20231224 (Metz, 2024) and labeled based on actions taken in each scenario. The datasets include fields *datetime*, *timestamp_desc*, *source*, *source_long*, *message*, *parser*, *display_name*, and *tag*.

Tables 4 and 5 show that actions in each dataset are similar except for file names, directory paths, file types, and duration. For example, in the training dataset $S1$, user account *training* executes Chrome, searches for "river otter pictures," and downloads a JPG file saved to the default Downloads directory. Whereas in the test dataset $S1'$, user account *testing* executes Chrome, searches for "arctic fox pictures," and downloads a PNG file saved to the default Downloads directory. Traces are labeled with class 1 (relevant) if they support the preponderance of evidence

Table 4

Labeled training datasets (duration), feature vector class distribution, and simulated action description by scenario.

Scenario	Class 0/1	Simulated Actions
S1 (3min)	10,468/135	Download <i>png</i> from internet
S2 (3min)	7,903/50	Create Notepad <i>txt</i>
S3 (4min)	8,726/148	Execute Paint3D, modify <i>png</i>
S4 (4min)	8,152/25	Move <i>png</i>
S5 (9min)	16,801/182	Upload <i>jpg</i> to Google Docs
S6 (5min)	22,695/138	Interact with GMail, send <i>log</i>
S7 (3min)	8,155/9	Transfer <i>png</i> to USB
S8 (5min)	8,242/30	Transfer <i>png</i> from USB
S9 (8min)	14,079/292	Download/edit/save/move/del
S10 (2min)	11,149/14	Delete <i>png</i>

Note: *png*, *txt*, *jpg*, and *log* represent file extensions.

Table 5

Labeled test datasets (duration), feature vector class distribution, and simulated action description by scenario.

Scenario	Class 0/1	Simulated Actions
S1' (2min)	19,429/119	Download <i>jpg</i> from internet
S2' (2min)	24,304/78	Create WordPad <i>rtf</i>
S3' (4min)	10,356/93	Execute MSPaint, modify <i>jpg</i>
S4' (4min)	8,301/33	Move a <i>jpg</i>
S5' (8min)	13,595/157	Upload <i>jpg</i> to Google Docs
S6' (5min)	22,301/118	Interact with GMail, send a <i>txt</i>
S7' (4min)	9,676/71	Transfer <i>png</i> to USB
S8' (4min)	16,802/40	Transfer <i>png</i> from USB
S9' (9min)	19,711/239	Download/edit/save/move/del
S10' (2min)	9,962/24	Delete <i>jpg</i>

Note: *jpg*, *rtf*, *png*, and *txt* represent file extensions.

Table 6

DTI model building confusion matrix data.

Scenario	TN	FP	FN	TP	Precision	Recall
S1'	19,420	9	0	119	0.93	1.00
S2'	24,293	11	0	78	0.87	1.00
S3'	10,356	0	0	93	1.00	1.00
S4'	8,301	0	0	33	1.00	1.00
S5'	13,586	9	0	157	0.98	1.00
S6'	22,301	0	0	118	1.00	1.00
S7'	9,676	0	0	71	1.00	1.00
S8'	16,799	3	0	40	1.00	1.00
S9'	19,711	0	0	239	1.00	1.00
S10'	9,962	0	0	24	1.00	1.00
Avg.	15,441	3	0	97	0.98	1.00

Table 7

DTI model building and Autopsy comparison results.

Scenario	DTI				Autopsy
	Duration (minutes)	EK Rules	F1	Steps, Time	Steps, Time
S1'	2m	32	0.96	5, 3m	15, 15m
S2'	2m	32	0.93	5, 4m	16, 10m
S3'	4m	31	1.00	4, 3m	16, 9m
S4'	4m	23	1.00	4, 2m	11, 7m
S5'	8m	32	0.97	5, 6m	14, 13m
S6'	5m	31	1.00	4, 4m	17, 12m
S7'	4m	21	1.00	4, 2m	11, 8m
S8'	4m	24	0.96	5, 3m	9, 5m
S9'	9m	38	1.00	4, 5m	13, 12m
S10'	2m	21	1.00	4, 1m	10, 7m
Avg.	4m	29	0.98	4, 3m	13, 10m

based on the user's actions and as class 0 otherwise. Studying dataset S1, the training user generated 15 traces while searching for pictures of river otters and created 21 traces during the subsequent PNG download.

Each dataset is divided into ten scenarios based on the recorded date-time during each simulation. Although user actions were similar during training and testing simulations, the total number of traces captured varied significantly due to background processes running on each workstation. For instance, notice how S1' has nearly twice as many traces as the training dataset S1, and S2 is only 30% the size of S2'. This imbalance is expected in laboratory and real-world environments and does not impact the efficacy of the DTI system.

We apply expert knowledge and string searches to identify traces associated with targeted activity during the labeling process. One advantage of using a binary classifier to model relevant traces created by target events is that all traces are initially irrelevant, and only those with relevant patterns are changed to class 1. This property enables an intuitive approach for experts to interact with metadata to create labeled datasets.

The data collection process produced two sets of labeled datasets of temporally ordered, semi-structured metadata traces: S1 – S10 for training and S1' – S10' for testing.

4.2. Results

The mean build time for accepted models is 19 minutes (100K iterations), demonstrating a reasonably fast build process for the activities modeled in this study. However, faster build times are possible by reducing datasets to use unique rules and oversampling both classes to a desired level. Further, build time was directly proportional to iterations and did not significantly change with feature size.

Tables 6 and 7 show the results for all models trained using DTI. Class imbalance in training data did not impact the model's ability to predict accurately as greater than 99% of all traces are irrelevant. Oversampling with replacement applied to training datasets did not negatively affect

the model. These results are reflected in perfect recall and an average F1 score of 0.98 across all scenarios.

False positives are present, with an average of three occurring across all scenarios. To determine the cause of these misclassifications, we analyzed the datasets for the models that produced false positives (S1, S2, S5, S8). In all datasets, identical rules were found in both classes, also known as "cross-loaded rules". For instance, the nine false positives in scenario one occurred because feature vectors for these traces only include two features, *chrome_download* and *logged_in*. Since both relevant and irrelevant traces include these two intervals, and no other characteristics are present to determine the difference, they were misclassified. These errors can be mitigated by adding characteristic EK rules to distinguish between classes but require discernable patterns for each class. If no patterns exist in the dataset, this type of misclassification is considered an irreducible error.

Regarding user experience, DTI was faster to use than Autopsy in correctly identifying labeled relevant traces. Where Autopsy requires 10 minutes and 13 steps to tag all traces, DTI requires 3 minutes and 4 steps—a 70% reduction in both metrics. EK rule development time is not included in this process as it varies based on the practitioner's experience, the complexity of the metadata traces under inspection, and discernible patterns found during analysis.

Fig. 8 shows accuracy and generality over iterations. During each iteration, the learned solution set evolves using offspring from the GA, and population size changes using the *Fitness* parameter and pruning. The learned rule population increases until approximately 20,000 iterations when the sum of all rule numerosities reaches 6,000, and only classifiers with the highest *Fitness* scores are kept in the population.

The maximum micro-classifier size, shown as the MacroPop size in Fig. 8 is set using the hyperparameter *N* and impacts the overall size of the solution set by setting a limit to the number of classifiers kept during each learning cycle. Recall how the maximum micro classifier population is the sum of classifier numerosities (number of copies). Ideally, the fittest classifiers occur multiple times, and the micro-population

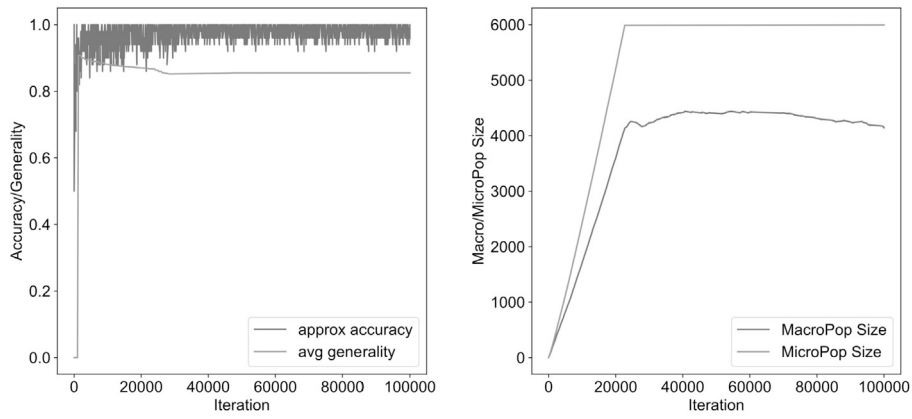


Fig. 8. Model accuracy and generality (left) and learned rule population (MicroPop) and the sum of classifier numerosities (MacroPop) as iterations increase (right) for $S9'$.

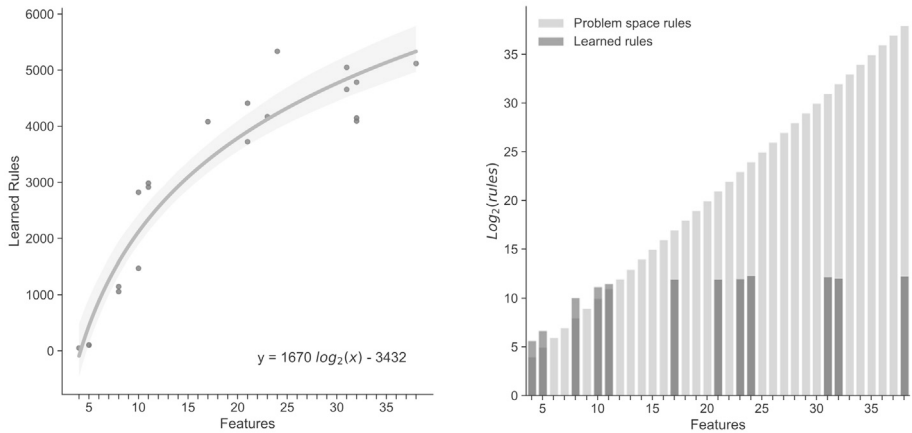


Fig. 9. Learned rules versus features (EK Rules) for Windows 10 atomics from Table 7 fit with $\log_2(x)$ transform and 95% confidence interval (left). Problem space and learned rules $\log_2$ versus feature size comparison (right).

size (the number of learned rules in the final model) decreases as rule generality and numerosity increase. Thus, the model for $S9'$ has approximately 4,000 rules whose numerosities sum to 6,000.

Fig. 9 shows how learned rules scale logarithmically as feature size increases, even with scenario nine, where the learned rule problem space is 2^{38} . The learned rule population of all models tested fell between 3,723 – 5,336 and, on average, was less than 0.05% of the overall problem space. Since MLCs rely on reinforcement learning, fewer iterations naturally lead to fewer GA operations and fewer offspring. In contrast, the hyperparameters N (maximum micro-classifier size) and v (*Fitness* power parameter) do not directly affect the number of learned rules, as they primarily influence the overall rule count and the importance of accuracy.

5. Discussion

This study presents an expert-focused temporal metadata tool, DTI, designed to locate and group corroborating digital traces of targeted activity. DTI enhances human decision-making in analysis tasks and provides an explainable AI mechanism to store and retrieve platform-specific knowledge using an intuitive, rule-based EK framework. The research questions, ‘RQ1 - Can accurate and explainable rule-based models be produced using simulated activities on a Windows 10 Workstation?’ and ‘RQ2 - How do these models compare to manual Autopsy analysis concerning the required steps and time?’, guide our evaluation of the DTI system. RQ1 was answered by designing, implementing, and evaluating the EK Framework and DTI system using simulated Windows 10 scenarios. Results show that the DTI tool can generate accurate and ex-

plainable rule-based models by leveraging expert knowledge to abstract complex events in temporal metadata. To address RQ2, we compared models trained with DTI to manual analysis of the same test datasets conducted with Autopsy, showing a 70% reduction in time and the number of steps required to identify targeted activities.

5.1. Key findings

One of this study’s key findings is that DTI’s use of rule-level classifiers enables it to generalize patterns found in traces to create features with and without an implied duration that can discern complexity in overlapping and interleaved traces. Additionally, the system is end-to-end explainable because features originate from a flexible, YARA-like rule structure, the rule-level learning process follows a “survival of the fittest” concept, and the final solution (model) can be summarized by reading the expert-defined feature names that are *True* and *False* for a given encoded trace. These results suggest that DTI can reduce the time needed to investigate temporal metadata traces by automatically identifying relevant metadata signatures, enabling analysts to focus on high-priority traces. Further, the components of the system are explainable by experts who can discuss their process of identifying trace evidence during legal proceedings.

DTI provides distinct advantages over traditional tools. Using EK rules, it is customizable to any semi-structured (ASCII delimited) temporal metadata dataset where users can identify patterns and create *interval* and *characteristic* abstractions. This capability is essential in digital forensics and other fields where temporal data is abundant and customizable decision support tools are either unavailable or inacces-

sible due to financial or security constraints. Another unique aspect of DTI is that it eliminates the need for retraining when users modify a rule, such as by updating a string or altering a regular expression. For example, modifying a feature that identifies a pattern in a URL with specific properties by changing a regular expression or string does not require retraining the model because only the output of the encoding process changed, not the learned rule set.

5.2. Limitations and challenges

In contrast, the proposed system has several limitations that may impact its effectiveness. The first and most significant limitation is that EK rule development requires expert knowledge, an understanding of the underlying cause of targeted activity, and distinct discernible patterns within the metadata. Thus, the system is ineffective without an experienced expert understanding of the platform and the ability to identify patterns within temporal metadata. Additionally, the 55 EK rules created during this study were customized to model the targeted activities in each scenario and would not likely model all potential scenarios on a Windows 10 workstation. However, as shown in Table 3, after creating the initial 14 EK rules for *S1* as a starting point, approximately 5 per model were necessary to train an accurate model.

Furthermore, the ‘curse of dimensionality’ poses a challenge, as many features can lead to large models that require more EK rules, training data, and iterations. While this is a valid concern, modeling events like those presented in this study required less than 30 features to classify traces accurately. Moreover, reduced models that include only the minimum number of features to explain the targeted activity are also effective.

Concerning training and model maintenance, the system relies on expert-labeled training data, which may not always be readily available. However, such data can be obtained from samples of real-world incidents and investigations or simulated in controlled lab environments, as demonstrated in this study. Adding a new EK rule requires retraining because the existing model will not understand how the new feature interacts with existing features. Another challenge during training is the occurrence of “cross-loaded rules”, where an encoded trace appears in both class 0 and class 1. In such cases, the rule is considered ambiguous and assigned to class 1, as the user has manually labeled at least one trace with this encoding. Thus, creating EK rules for targeted activity that do not intersect with patterns found in irrelevant traces will avoid false positives created by cross-loaded rules.

5.3. Scalability

Digital forensics commonly involves managing large sets of traces from multiple devices. DTI faces three potential bottlenecks when working with large-scale data: encoding traces into feature vectors, training models, and predicting unseen traces. Though all data processing in the DTI prototype applies vector operations to reduce unnecessary operations, each trace must be encoded using EK rules. This process has the time complexity of $O(NM)$, where N is the number of unique traces and M is the number of EK rules.

Besides rule encoding, rule-level classifiers are efficient for training and prediction using DTI. After encoding the training data into feature vectors, only a single instance of each is required to train a model. If a balanced dataset is desired, the minority class can be oversampled with replacement; nonetheless, MLCSs can train an accurate model with imbalanced data as long as the rules sufficiently distinguish each class. While balancing can reduce the number of iterations needed to reach an accurate solution, it is not necessary.

Similarly, all traces must be encoded first when predicting labels for unseen datasets. However, only one prediction per unique feature vector is necessary. To reduce the number of predictions required, first reduce the unseen encoded dataset to unique feature vectors, then predict a

label for each and map the result to the original dataset using a vector operation with $O(N)$ complexity.

6. Conclusions and future work

The methodology successfully labeled metadata traces from various scenarios on a Windows 10 workstation, including common user actions such as downloading, modifying, and deleting files, even in complex situations with overlapping applications. We also see that MLCS models require little training data, handle imbalanced classes well, and are explainable since models consist of a set of rules with features named after expert-defined rules. Additionally, the EK Framework provides a novel method for extracting and encoding machine learning features and enables collaboration within teams or with a broader community. Further, compared to Autopsy, all trained models require less time and effort to identify the same events.

Unfortunately, false positives exist due to interleaved and indiscernible patterns. While false positives can be reduced by combining *interval* and *characteristic* EK rules, they ultimately depend on how well patterns can be identified in the underlying data.

Continued development of explainable and customizable automation tools is critical, as trustworthy AI is tied to human social organization behavior that is becoming the standard for AI applications. Future work is required to test the efficacy of this methodology on other digital platforms, e.g., Linux, MacOS, and mobile devices, to extend DTI to support multi-class classification, and conduct a use case study with digital forensics experts.

CRedit authorship contribution statement

Michael C. Todd: Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Gilbert L. Peterson:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. The views expressed in this paper are those of the authors and do not represent the views or policies of the United States Air Force, or the Department of Defense.

Data availability

The datasets used in this study, along with the DTI prototype, are available on github at https://github.com/TheCodeheadMT/dti_prototype. Researchers and practitioners can access these resources for further analysis and replication of results.

References

- Adderley, N., Peterson, G., 2020. Interactive temporal digital forensic event analysis. In: *Advances in Digital Forensics XVI: 16th IFIP WG 11.9 International Conference*. New Delhi, India, January 6–8, 2020, Revised Selected Papers, vol. 589, pp. 39–55.
- Agrawal, R., Srikant, R., 1995. Mining sequential patterns. In: *Proceedings of the Eleventh International Conference on Data Engineering*, pp. 3–14.
- Alvarez, V., 2013. Yara: the pattern matching Swiss knife for malware researchers. (Version 4.5.0) [Computer software]. <https://github.com/VirusTotal/yara>.
- Berggren, J., Peterson, A., 2023. Timesketch: open source collaborative forensic timeline analysis tool. (Version 20240328) [Computer software]. <https://timesketch.org/>.

- Carrier, B., 2013. The sleuth kit (tsk) and autopsy: open source digital forensics tools. (Version 4.21.0). [Computer software]. <https://sleuthkit.org/>.
- Costantini, S., De Gasperis, G., Olivieri, R., 2019. Digital forensics and investigations meet artificial intelligence. *Ann. Math. Artif. Intell.* 86, 193–229.
- Debinski, M., Breitinger, F., Mohan, P., 2019. Timeline2gui: a log2timeline csv parser and training scenarios. *Digit. Investig.* 28, 34–43. <https://doi.org/10.1016/j.diin.2018.12.004>.
- Efron, B., Tibshirani, R., 1994. *An Introduction to the Bootstrap*, 1st ed. Chapman and Hall/CRC.
- Grimm, P.W., Grossman, M.R., Cormack, G.V., 2021. Artificial intelligence as evidence. *Northwest. J. Technol. Intellect. Prop.* 19, 9.
- Guidance-Software, 2023. Encase forensic: the fastest, most comprehensive forensic solution available. (Version 7.10). [Computer software]. <https://www.opentext.com/products/encase-forensic>.
- Holland, J.H., Booker, L.B., Colombetti, M., Dorigo, M., Goldberg, D.E., Forrest, S., Riolo, R.L., Smith, R.E., Lanzi, P.L., Stolzmann, W., Wilson, S.W., 2000. What is a learning classifier system? https://doi.org/10.1007/3-540-45027-0_1.
- McKinney, W., 2010. Data structures for statistical computing in python. In: *Proceedings of the 9th Python in Science Conference*, pp. 56–61.
- Merriam-Webster, 2024. Meta - definition and examples. Dictionary Lookup. Retrieved from <https://www.merriam-webster.com/dictionary/meta>.
- Metz, J., 2024. Log2timeline-plaso. (Version 20231224). [Computer software]. <https://github.com/log2timeline/plaso>.
- Nisioti, A., Loukas, G., Laszka, A., Panaousis, E., 2021. Data-driven decision support for optimizing cyber forensic investigations. *IEEE Trans. Inf. Forensics Secur.* 16, 2397–2412. <https://doi.org/10.1109/TIFS.2021.3054966>.
- Okolica, J.S., 2017. Temporal event abstraction and reconstruction. *Theses and Dissertations*.
- Okolica, J.S., Peterson, G.L., Mills, R.F., Grimaila, M.R., 2020. Sequence pattern mining with variables. *IEEE Trans. Knowl. Data Eng.* 32, 177–187.
- Oliphant, T., 2006. *Guide to NumPy*, 1st ed. Trelgol Publishing, USA.
- Orriols, A., Bernadó-Mansilla, E., 2005. The class imbalance problem in learning classifier systems: a preliminary study. In: *Proceedings of the 7th Annual Workshop on Genetic and Evolutionary Computation*. Association for Computing Machinery, New York, NY, USA, pp. 74–78.
- Paxson, V., 1999. Bro: a system for detecting network intruders in real-time. *Comput. Netw.* 31, 2435–2463. [https://doi.org/10.1016/S1389-1286\(99\)00112-7](https://doi.org/10.1016/S1389-1286(99)00112-7).
- Roesch, M., 1999. Snort - lightweight intrusion detection for networks. In: *LISA '99: Proceedings of the 13th USENIX Conference on System Administration*. USENIX Association, pp. 229–238.
- Shim, J.P., Warkentin, M., Courtney, J.F., Power, D.J., Sharda, R., Carlsson, C., 2002. Past, present, and future of decision support technology. *Decis. Support Syst.* 33, 111–126.
- Tanenbaum, A., 2009. *Modern Operating Systems*. Pearson Education, Inc., London.
- Turnbull, B., Randhawa, S., 2015. Automated event and social network extraction from digital evidence sources with ontological mapping. *Digit. Investig.* 13, 94–106. <https://doi.org/10.1016/j.diin.2015.04.004>.
- Urbanowicz, R.J., Browne, W., 2015. Introducing rule-based machine learning: a practical guide. In: *GECCO 2015 - Companion Publication of the 2015 Genetic and Evolutionary Computation Conference*.
- Urbanowicz, R.J., Moore, J.H., 2009. Learning classifier systems: a complete introduction, review, and roadmap. *J. Artif. Evol. Appl.* 2009, 1–25. <https://doi.org/10.1155/2009/736398>.
- Urbanowicz, R.J., Moore, J.H., 2015. Exstracs 2.0: description and evaluation of a scalable learning classifier system. *Evolut. Intell.* 8, 89–116. <https://doi.org/10.1007/s12065-015-0128-8>.
- Urbanowicz, R.J., Granizo-Mackenzie, A., Moore, J.H., 2012. An analysis pipeline with statistical and visualization-guided knowledge discovery for Michigan-style learning classifier systems. *IEEE Comput. Intell. Mag.* 7, 35–45. <https://doi.org/10.1109/MCI.2012.2215124>.
- Villafane, R., Hua, K.A., Tran, D., Maulik, B., 2000. Knowledge discovery from series of interval events. *J. Intell. Inf. Syst.* 15, 71. Copyright - Copyright (c) 2000 Kluwer Academic Publishers Last updated - 2023-11-27.
- Wilson, S.W., 1995. Classifier fitness based on accuracy. *Evolut. Comput.* 3, 149–175. <https://doi.org/10.1162/evco.1995.3.2.149>.